

Asymptotic Analysis and Growth Rates

From exact operation counts to asymptotic growth

FROM LECTURE 6

$C(n) \rightarrow$ growth shape

As n grows, exact operation counts can be summarized by their long-run growth pattern.

$\log n$

slow

n

steady

n^2

fast

2^n

explodes

What should you be able to do by the end?

By the end of this lecture, you should be able to interpret and compare common growth rates.

SEE GROWTH

explain what happens as n becomes large

SIMPLIFY

identify dominant terms and ignore non-dominant detail

RECOGNIZE

spot common loop-growth patterns

USE Θ

state a tight growth class when appropriate

DISTINGUISH

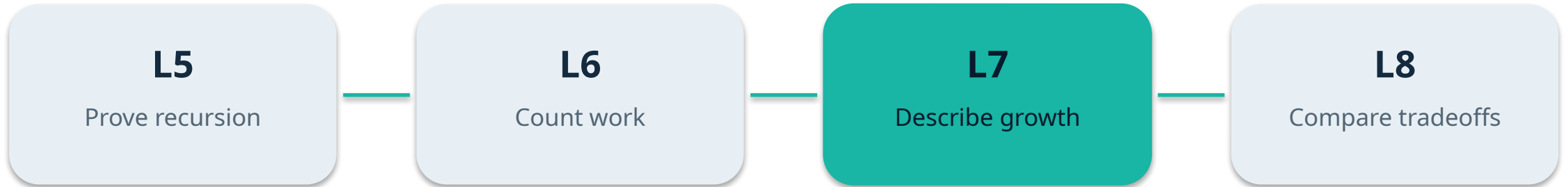
interpret O , Ω and Θ correctly

CONNECT

turn Lecture 6 counts into asymptotic classes

Where Lecture 7 fits

Lecture 6 counted work. Lecture 7 describes how that work scales.



$$C(n) = n(n-1)/2 \rightarrow \Theta(n^2)$$

Lecture 6: “How many?”

Lecture 7: “How fast does that number grow?”

Start with numbers: which column grows fastest?

First compare the values and how quickly they increase.

n	$\log_2 n$	n	n^2
10	≈ 3.3	10	100
100	≈ 6.6	100	10,000
1,000	≈ 10	1,000	1,000,000
1,000,000	≈ 20	1,000,000	1,000,000,000,000

Notice the SHAPE

$\log n$ barely moves, n grows steadily, n^2 grows dramatically faster.

What do we mean by “growth”?

Growth asks how the amount of work changes when the input becomes larger.

$n = 10$

linear work

10 basic operations

$n = 100$

linear work

100 basic operations

$n = 1,000$

linear work

1,000 basic operations

Question: If input becomes 10× larger, what happens to the work?

That relationship — not the exact stopwatch time — is what asymptotic analysis describes.

Exact count first. Growth class second.

We are not replacing $C(n)$. We are summarizing its long-run shape.

1 COUNT

Choose a basic operation and derive $C(n)$.
Example: $C(n)=3n^2+5n+7$

2 LOOK

Ask which part grows fastest as n becomes large.

3 CLASSIFY

Describe the growth shape.
Here: quadratic $\rightarrow \Theta(n^2)$

Keep the exact count when you need detail. Use the growth class when you need comparison.

Dominant term — start with an easy one

$$C(n)=3n+5$$

$$C(n) = 3n + 5$$

For small n

the +5 may be noticeable.

For large n

the growing part $3n$ dominates.

Growth shape

$3n$ and n have the same basic shape.

$3n+5 \rightarrow$ linear growth

Now a quadratic expression

$$C(n)=4n^2+3n+20$$

$$4n^2 + 3n + 20$$

Which term grows fastest?

20

constant

3n

linear

4n²

quadratic

Fastest-growing term: n^2 → quadratic growth

Constants change scale. Exponents change growth.

This is the intuition behind asymptotic simplification.

$2n$ vs n

$2n$ is always exactly twice n .
The ratio never grows.

$1000n$ vs n

A huge constant can matter in practice, but the growth SHAPE is still linear.

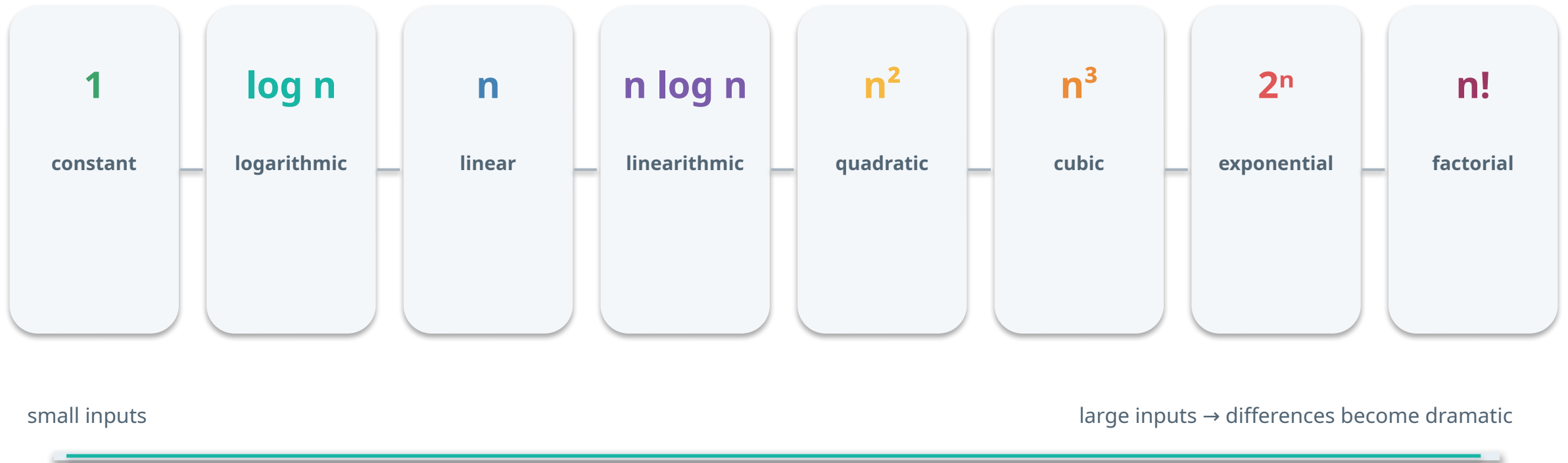
n^2 vs n

The ratio $n^2/n = n$ keeps growing.
Different exponent $\rightarrow$ different class.

Scale $\neq$ growth class

The growth ladder

From slowest-growing to fastest-growing for large n .



Group the ladder instead of memorizing eight names

Think in families.

VERY SCALABLE

$1 \cdot \log n \cdot n$
Growth stays relatively controlled.

MIDDLE

$n \log n$
Common in efficient sorting/divide-and-conquer.

GETS EXPENSIVE

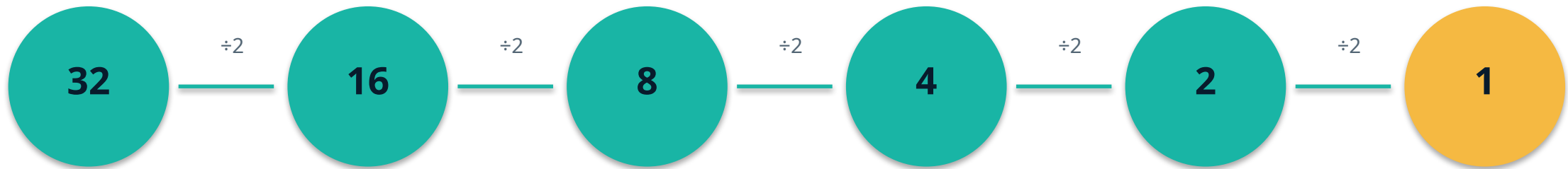
$n^2 \cdot n^3$
Nested work grows rapidly.

EXPLODES

$2^n \cdot n!$
Even moderate n can become infeasible.

Repeated halving: count the arrows

Start with $n = 32$.



How many halvings?

5 steps

because $32 = 2^5$

Why is the number of steps $\log_2 n$?

Now turn the picture into mathematics.

After 1 step

$$n / 2$$

After 2 steps

$$n / 2^2$$

After k steps

$$n / 2^k$$

Stop when

$$n / 2^k = 1$$

$$n / 2^k = 1 \rightarrow n = 2^k \rightarrow k = \log_2 n$$

Doubling gives the same pattern

Not only halving — repeated doubling is logarithmic too.

PSEUDOCODE

```
i ← 1  
while i < n  
    i ← 2 × i
```

1 → 2 → 4 → 8 → 16 → 32 → ...

After k doublings

$$i = 2^k$$

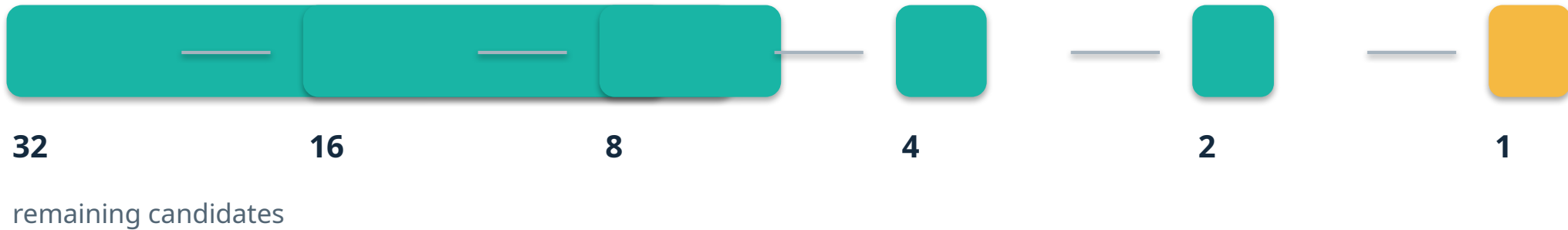
Reach n when

$$2^k \approx n$$

Therefore: **$\Theta(\log n)$**

Binary Search = repeated halving

Every comparison discards about half of the remaining search space.



Key idea

Binary Search does not remove one item per comparison. It removes roughly HALF.

1,000,000 items: linear vs. binary search

The difference becomes dramatic for large inputs.

LINEAR SEARCH

$\approx 1,000,000$

comparisons in the worst case

BINARY SEARCH

≈ 20

halving decisions

$$\log_2(1,000,000) \approx 19.93$$

Why the base of the logarithm usually disappears

Changing the base only introduces a constant multiplier.

$$\log_{10} n = \log_2 n / \log_2 10$$

The denominator

$\log_2 10$ is just a constant.

So asymptotically

$\log_2 n$, $\log_{10} n$ and $\ln n$ share the same growth class.

We usually write simply $\Theta(\log n)$

Θ means “same growth order”

Use Θ when the function is tightly bounded by that growth shape.

$$C(n)=3n+5$$

linear shape
 $\rightarrow \Theta(n)$

$$C(n)=\frac{1}{2}n^2-\frac{1}{2}n$$

quadratic shape
 $\rightarrow \Theta(n^2)$

$$C(n)=n^3$$

cubic shape
 $\rightarrow \Theta(n^3)$

Plain English: the exact function and the comparison function grow at the same asymptotic rate.

Lecture 6 counts $\rightarrow \Theta$ classes

Same algorithms, now summarized by growth.

Lecture 6 pattern	Exact count $C(n)$	Growth class
SUM / MAX	n or $n-1$	$\Theta(n)$
Matrix addition ($n \times n$)	n^2	$\Theta(n^2)$
Unordered pairs	$n(n-1)/2$	$\Theta(n^2)$
Matrix multiplication	n^3	$\Theta(n^3)$
Halving loop	$\log_2 n$	$\Theta(\log n)$

This is the main bridge from Lecture 6 to Lecture 7.

A tighter picture: Θ traps $f(n)$ between two copies of $g(n)$

Formal definition of tight asymptotic growth.

$$c_1g(n) \leq f(n) \leq c_2g(n)$$

for all $n \geq n_0$

c_1 and c_2

fixed positive multipliers

n_0

the point after which the
bounds keep holding

Meaning

same long-run growth
order

Do not memorize the symbols without the picture.

Big-O: an eventual upper bound

Think: “f(n) does not outgrow this ceiling.”

UPPER CEILING

$$f(n) \leq c \cdot g(n)$$

eventually

Plain English

After some point, f(n) stays at or below a constant multiple of g(n).

Example: $3n+2 \in O(n)$

Formal Big-O — now the notation has meaning

There exist constants $c > 0$ and n_0 such that...

$$0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

$f(n)$

the function we are bounding

c

a positive constant multiplier

$g(n)$

the comparison growth
function

n_0

the threshold after which it
holds

“Eventually” is why asymptotic analysis can ignore small- n irregularities.

Worked example: prove an upper bound

Show $3n+2 \in O(n)$.

Start

$$3n + 2$$

For $n \geq 1$

$$2 \leq 2n$$

Therefore

$$3n + 2 \leq 5n$$

Choose

$$c = 5, n_0 = 1$$

Hence $3n+2 \in O(n)$

Big-O can be true but loose

Upper bound does not automatically mean “best description.”

$$f(n)=3n+2$$

$O(n)$

tight upper bound

$O(n^2)$

true but loose

$O(n^3)$

also true but looser

So what should we usually report?

When we mean the exact asymptotic growth order, Θ is more informative.

Big-Ω: an eventual lower bound

Think: “f(n) grows at least this fast.”

LOWER FLOOR

$$f(n) \geq c \cdot g(n)$$

eventually

Example

$3n+2 \geq 3n$ for every $n \geq 0$.
So $3n+2 \in \Omega(n)$.

Ω gives a floor. O gives a ceiling.

Θ = upper bound + lower bound

When both use the same $g(n)$, the bound is tight.

$O(g(n))$ — eventual ceiling

$\Theta(g(n))$ — trapped in the same growth order

$\Omega(g(n))$ — eventual floor

If $f(n) \in O(g(n))$ and $f(n) \in \Omega(g(n))$, then **$f(n) \in \Theta(g(n))$**

Big-O does NOT mean “worst case”

Case analysis and asymptotic notation answer different questions.

CASE ANALYSIS

Best / worst / average asks: which input situation are we discussing?

ASYMPTOTIC NOTATION

O / Ω / Θ asks: what kind of bound or growth description are we using?

Example — Linear Search worst case: $\Theta(n)$

The “worst case” came from the input position of the target; Θ describes how that worst-case count grows.

Same algorithm, different cases — same notation vocabulary

Case first, then classify the growth.

Case	Comparison count	Asymptotic growth
Best	1	$\Theta(1)$
Worst	n	$\Theta(n)$
Average successful (uniform position)	$(n+1)/2$	$\Theta(n)$

The case changes the count. The asymptotic class summarizes how that count grows.

Three basic patterns to recognize quickly

Do not memorize “one loop = n.” Look at how the variable changes.

CONSTANT

```
x ← A[0]
```

$\Theta(1)$

HALVING

```
while n > 1  
  n ← n/2
```

$\Theta(\log n)$

FULL SCAN

```
for i ← 1 to n  
  operation
```

$\Theta(n)$

Nested loops: count what EACH loop does

Two loops do not automatically mean n^2 .

$$n \times \log n$$

outer n times
inner halves each time

$$\Theta(n \log n)$$

$$n \times n$$

two full n loops

$$\Theta(n^2)$$

$$n \times n \times n$$

three full n loops

$$\Theta(n^3)$$

Sigma is just another way to write the loop count

Rectangular $n \times n$ loop.

LOOP

```
for i ← 1 to n
  for j ← 1 to n
    operation
```

$$C(n) = \sum_{i=1}^n \sum_{j=1}^n 1$$

$$= \sum_{i=1}^n n$$

$$= n^2 \rightarrow \Theta(n^2)$$

Triangular nested loop: inner work changes each time

This is why “two loops = n^2 ” is only a rough pattern.

LOOP

```
for i ← 1 to n-1  
  for j ← i+1 to n  
    operation
```

$$C(n) = (n-1) + (n-2) + \dots + 1$$

$$= \sum_{k=1}^{n-1} k$$

$$= n(n-1)/2$$

$$\rightarrow \Theta(n^2)$$

Matrix addition: connect loops, sigma and growth

For an $n \times n$ matrix.

MATRIX ADDITION

```
for i ← 0 to n-1
  for j ← 0 to n-1
    C[i][j] ← A[i][j] + B[i][j]
```

Basic operation

one scalar addition

Exact count

$C(n) = n^2$ additions

$$\sum_{i=0}^{n-1} \sum_{j=0}^{n-1} 1 = n^2$$

Therefore $\Theta(n^2)$

A nested loop can be $n \log n$

Inspect the bounds and update rules.

PATTERN

```
for i ← 1 to n
  j ← n
  while j > 1
    operation
    j ← j / 2
```

Outer loop

n iterations

Inner loop

$\log n$ halving steps

$$C(n) = n \cdot \log n$$

$$\rightarrow \Theta(n \log n)$$

n vs. n log n vs. n²

Small differences become huge at scale.

n	n	$n \log_2 n$	n^2
16	16	64	256
1,024	1,024	10,240	1,048,576
1,048,576	$\approx 1.05 \times 10^6$	$\approx 2.10 \times 10^7$	$\approx 1.10 \times 10^{12}$

Interpretation

n log n grows more than linear, but dramatically less than quadratic for large n.

Exponential and factorial growth explode

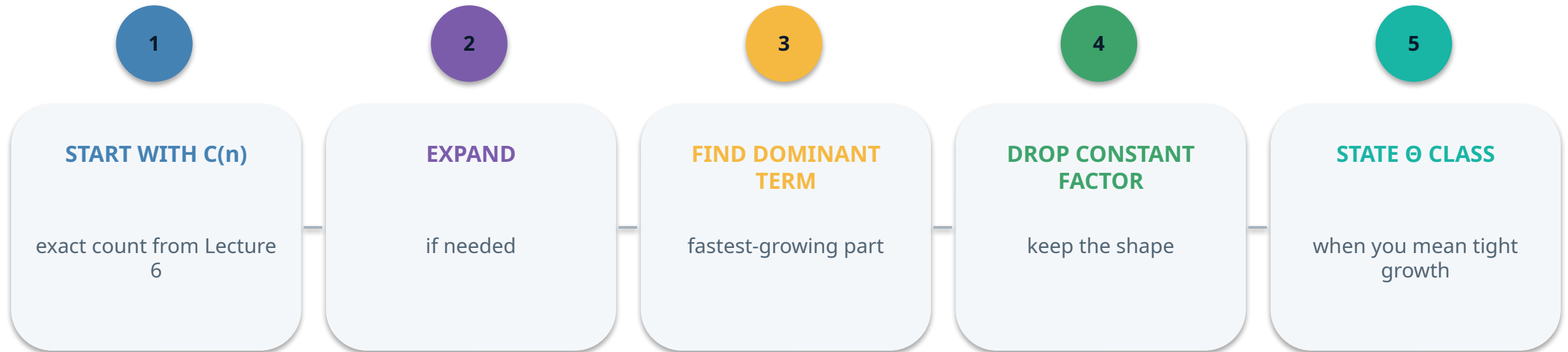
These often appear when brute force enumerates subsets or permutations.

n	2^n	$n!$
10	1,024	3,628,800
20	1,048,576	$\approx 2.43 \times 10^{18}$
30	$\approx 1.07 \times 10^9$	$\approx 2.65 \times 10^{32}$

$$\log n \ll n \ll n \log n \ll n^2 \ll 2^n \ll n!$$

A simple recipe for every Lecture 7 problem

Use the same five steps repeatedly.



Example: $n(n-1)/2 \rightarrow \frac{1}{2}n^2 - \frac{1}{2}n \rightarrow n^2 \rightarrow \Theta(n^2)$

Four common misconceptions

Check each statement carefully and identify the correct reasoning.

"One loop means $\Theta(n)$."

A loop that halves its variable is $\Theta(\log n)$.

"Two loops mean $\Theta(n^2)$."

Bounds may produce $n \log n$, triangular counts, etc.

"Big-O means worst case."

Big-O is an upper bound; worst case is an input case.

"Drop every term."

Keep the dominant growth term; do not claim algebraic equality.

Practice A — classify the growth

First simplify mentally, then state the tight class.

1

$$C(n) = 7n + 40$$

2

$$C(n) = 3n^2 + 2n + 10$$

3

$$C(n) = n(n-1)/2$$

4

$$C(n) = 5\log_2 n + 100$$

Think: dominant term $\rightarrow$ constant factor $\rightarrow \Theta$ class.

Practice A — check your reasoning

Say WHY, not only the symbol.

$$7n+40$$

n dominates the constant

$$\Theta(n)$$

$$3n^2+2n+10$$

highest power is n^2

$$\Theta(n^2)$$

$$n(n-1)/2$$

$$= \frac{1}{2}n^2 - \frac{1}{2}n$$

$$\Theta(n^2)$$

$$5\log_2 n + 100$$

constant factor/base do not change the class

$$\Theta(\log n)$$

Practice B — read the loop shape

Do not count loop keywords. Analyze the update rules.

A

```
A) i ← 1
   while i < n
       operation
       i ← 2i
```

B

```
B) for i ← 1 to n
    for j ← 1 to n
        operation
```

C

```
C) for i ← 1 to n
    j ← n
    while j > 1
        j ← j/2
```

A $\Theta(\log n)$ **B** $\Theta(n^2)$ **C** $\Theta(n \log n)$

Eight ideas to carry forward

Lecture 7 turns exact counting into a language for comparing growth.

01

START WITH $C(n)$

02

DOMINANT TERM

03

$\log n$ = HALVING

04

Θ = TIGHT GROWTH

05

O = UPPER BOUND

06

Ω = LOWER BOUND

07

CASES $\neq$ NOTATION

08

READ LOOP LIMITS

NEXT: Lecture 8 — Complexity Classes and Performance Tradeoffs